ClimateProcessingEngine

A Comprehensive Python Framework for Climate Data Processing

Amin Fazl Kazemi
aminfazlkazemi@gmail.com

Version 2.1
July 26, 2026

Contents

List of Figures

List of Tables

Listings

Chapter 1

Introduction

1.1 Overview

ClimateProcessingEngine is an open-source Python framework designed for processing large-scale climate datasets, performing statistical distribution fitting, and generating climatological summaries. The engine is optimized for station-wise time series data with support for multiple variables (temperature, precipitation, etc.) and handles missing data through robust window-based approaches.

1.1.1 Key Features

- **Block-oriented processing** – Handles millions of stations efficiently using configurable block sizes
- **Multi-distribution fitting** – Supports Normal, Skew-Normal, Bimodal, and Pearson Type III distributions
- **Parallel processing** – Optional multi-core execution for accelerated computation
- **Checkpoint recovery** – Automatic resumption from the last completed block
- **Zarr output** – Efficient, chunked, cloud-optimized storage format
- **Comprehensive validation** – Data integrity checks at multiple pipeline stages
- **Benchmarking** – Automatic selection of optimal block size based on system resources

1.1.2 Target Audience

- Climate scientists and meteorologists
- Environmental researchers
- Data engineers working with geospatial time series
- Graduate students in climate science

1.1.3 System Requirements

Table 1.1: Minimum System Requirements

Component	Requirement
Operating System	Windows 10/11, Linux, or macOS
Python	3.8 or higher
RAM	16 GB minimum (32 GB recommended)
Storage	100 GB free space (for intermediate and output data)
CPU	4 cores minimum (8+ recommended for parallel mode)

1.1.4 Dependencies

The following Python packages are required:

```
numpy>=1.24.0
pandas>=2.0.0
xarray>=2023.0.0
dask>=2023.0.0
rasterio>=1.3.0
geopandas>=0.14.0
cartopy>=0.22.0
matplotlib>=3.7.0
scipy>=1.10.0
scikit-learn>=1.3.0
shapely>=2.0.0
netcdf4>=1.6.0
zarr>=2.15.0
numba>=0.57.0
pyyaml>=6.0
psutil>=5.9.0 (optional, for RAM monitoring)
```

Listing 1.1: Required Dependencies

Chapter 2

Architecture Overview

2.1 System Architecture

The ClimateProcessingEngine follows a modular, pipeline-based architecture with clear separation of concerns. Figure **??** illustrates the overall system design.

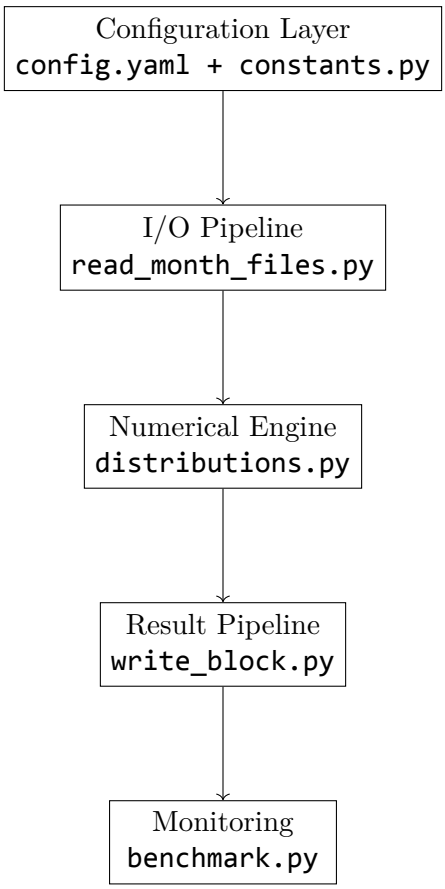

Figure 2.1: System Architecture Overview

2.2 Core Components

2.2.1 Configuration Layer (`constants.py + config.yaml`)

Centralizes all settings in a single YAML file, parsed by `constants.py`. This ensures consistent configuration across all modules.

2.2.2 I/O Pipeline (`io_pipeline/*.py`)

Responsible for reading input data from Zarr archives, assembling blocks, and validating loaded data.

2.2.3 Numerical Engine (`numerical_engine/*.py`)

Performs the core statistical analysis, including window extraction, distribution fitting, and result aggregation.

2.2.4 Result Pipeline (`result_pipeline/*.py`)

Validates and writes results to the output Zarr store.

2.2.5 Monitoring (`monitoring/*.py`)

Handles logging, checkpointing, and benchmarking.

2.2.6 Orchestrator (`orchestrator/*.py`)

Coordinates the execution flow, managing the block processing loop.

2.3 Data Flow

The data flow follows a linear pipeline:

1. **Load Phase**: Read raw data from input Zarr files for a block of stations
2. **Assembly Phase**: Arrange data into a 4D array (station, year, day, variable)
3. **Validation Phase**: Validate the assembled data against the data contract
4. **Analysis Phase**: For each station, extract windows and fit distributions
5. **Result Validation**: Validate analysis results before writing
6. **Write Phase**: Write validated results to the output Zarr store
7. **Checkpoint**: Record progress for recovery

Chapter 3

Installation and Setup

3.1 Installation

3.1.1 Clone the Repository

```
git clone https://github.com/AminFazlKazemi/ClimateProcessingEngine.git
cd ClimateProcessingEngine
```

Listing 3.1: Cloning the Repository

3.1.2 Create a Virtual Environment

```
# Windows
python -m venv venv
venv\Scripts\activate

# Linux/macOS
python3 -m venv venv
source venv/bin/activate
```

Listing 3.2: Creating Virtual Environment

3.1.3 Install Dependencies

```
pip install -r requirements.txt
```

Listing 3.3: Installing Dependencies

3.2 Configuration

3.2.1 Configuration File Structure

The `config.yaml` file contains all runtime settings:

```
# Paths
paths:
  calendar_file: K:/Temp/needed/calendar.txt
  checkpoint_file: checkpoint.csv
```

```yaml
  output_dir: I:/climatology_366_rolling
  output_zarr_name: climatology_stationwise_final.zarr
  zarr_base: K:/gozareshha/dr vazife/140504 - qc temp/zarr_yearly_monthly

# Time Range
years:
  start: 1369
  end: 1399
days: 366
window_days: 2

# Processing
block_size: 1000
cores: 6
use_parallel: false

# Variables
variables:
- tmin
- tmean
- tmax
fit_variable_index: 1

# Validation
validation:
  validate_after_load: true
  validate_before_write: true
  validate_every_n_blocks: 5

# Logging
logging:
  level: INFO
  log_file: climatology.log
  log_timestamps: true

# Benchmark
benchmark:
  enabled: true
  run_on_first_block: true
  test_block_sizes:
  - 500
  - 1000
  - 1500
  - 2000
  - 3000
```

Listing 3.4: Example config.yaml

3.2.2 Understanding Configuration Parameters

Paths

Table 3.1: Path Configuration Parameters

Parameter	Description
`calendar_file`	Path to the Persian calendar mapping file
`checkpoint_file`	Location for storing recovery checkpoint
`output_dir`	Directory where output Zarr will be saved
`output_zarr_name`	Name of the output Zarr file
`zarr_base`	Directory containing input Zarr files (organized as `YYYY_MM.zarr`)

Time Configuration

Table 3.2: Time Configuration Parameters

Parameter	Description
`years.start`	First year of the analysis (Persian calendar)
`years.end`	Last year of the analysis (inclusive)
`days`	Number of days in the year (365 or 366)
`window_days`	Days before/after each target day for window extraction

The window size is calculated as: `WINDOW_SIZE = 2 * window_days + 1`

Processing Configuration

Table 3.3: Processing Configuration Parameters

Parameter	Description
`block_size`	Number of stations per processing block
`cores`	Number of CPU cores to use when parallel processing is enabled
`use_parallel`	If true, enables multi-core processing

3.3 Preparing Input Data

3.3.1 Input Zarr Structure

The engine expects input Zarr files organized by year and month:

```
zarr_base/
--- 1369_01.zarr/
|   --- tmin (2D: day, point)
|   --- tmax (2D: day, point)
|   --- tmean (2D: day, point)
|   --- stationid (1D: point)
|   --- lat (1D: point)
|   --- lon (1D: point)
|   --- elev (1D: point)
--- 1369_02.zarr/
--- ...
```

Listing 3.5: Input Zarr Directory Structure

3.3.2 Calendar File Format

The calendar file (`calendar.txt`) maps Persian dates to day-of-year indices:

```
ShamsiDate  ShamsiDOY  GregorianDate  GregorianDOY
13690101    1          19900322       80
13690102    2          19900323       81
...
```

Listing 3.6: calendar.txt Format

Chapter 4

Core Modules Reference

4.1 Configuration Module (`constants.py`)

4.1.1 Overview

The `constants.py` module loads configuration from `config.yaml` and defines all constants used throughout the engine.

4.1.2 Key Constants

```
# Time
YEAR_START = 1369
YEAR_END = 1399
N_YEARS = 31
N_DAYS = 366
WINDOW_DAYS = 2
WINDOW_SIZE = 5              # 2*window_days + 1

# Variables
VARS = ['tmin', 'tmean', 'tmax']
N_VARS = 3
FIT_VARS = ['tmin', 'tmean', 'tmax']
N_OUTPUTS = 90               # 30 outputs per variable x 3 variables

# Paths
ZARR_BASE = "..."
OUTPUT_DIR = "..."
OUTPUT_ZARR = "..."

# Processing
BLOCK_SIZE = 1000
N_CORES = 6
USE_PARALLEL = False

# Validation thresholds
MAX_VALUES_PER_FIT = 155     # N_YEARS * WINDOW_SIZE
MIN_VALID_VALUES = 5
VALID_BEST_DIST = {-1, 0, 1, 2, 3}

# Data types
FLOAT_DTYPE = np.float32
```

```
INT_DTYPE = np.int32
```

Listing 4.1: Constants Defined

4.1.3 Configuration Functions

```
def load_config(path):
    """Load YAML configuration file"""
    with open(path, 'r', encoding='utf-8') as f:
        return yaml.safe_load(f)

def ensure_directories():
    """Create output directories if they don't exist"""
    os.makedirs(OUTPUT_DIR, exist_ok=True)

def validate_config():
    """Validate configuration values and raise errors on invalid settings
    """
    # Checks for valid year range, block size, etc.
```

Listing 4.2: Configuration Functions

4.2 Zarr Schema Module (`zarr_schema.py`)

4.2.1 Overview

The `zarr_schema.py` module defines the output schema for the Zarr store. It serves as the single source of truth for all output variables.

4.2.2 Variable Definitions

Each output variable is defined as a tuple: `(name, dtype, fill_value, description)`.

Table 4.1: Output Variables per Temperature Variable

Category	Variables	Count
Best Distribution	`best_dist`	1
Statistics	`mean, std, skewness, median, count`	5
Normal Parameters	`normal_p1, p2, loglik, aicc, bic`	5
Skew-Normal Parameters	`skew_p1, p2, p3, loglik, aicc, bic`	6
Bimodal Parameters	`bimodal_p1, p2, p3, p4, p5, loglik, aicc, bic`	8
Pearson III Parameters	`pearson_p1, p2, p3, loglik, aicc`	5
Total per variable		**30**
Total for 3 variables		**90**

Note: The actual implementation uses 30 outputs per variable (excluding `pearson_bic`), resulting in 87 total variables.

4.2.3 Key Functions

```
def create_zarr_store(output_path, n_stations, chunk_size=(366, 100)):
    """
    Create a Zarr store with all output variables.

    Parameters:
        output_path: str - Path to create the Zarr store
        n_stations: int - Total number of stations
        chunk_size: tuple - Chunk size for Zarr arrays (day, station)

    Returns:
        zarr.Group: The root group of the Zarr store
    """
    # Creates arrays for each variable with specified dtype and fill value

def add_coords_and_metadata(ds, station_ids, lons, lats, elevs):
    """
    Add coordinate variables and metadata to an xarray Dataset.

    Parameters:
        ds: xarray.Dataset - Dataset to augment
        station_ids: array - Station identifiers
        lons, lats, elevs: arrays - Geographic metadata

    Returns:
        xarray.Dataset: Dataset with coordinates and metadata
    """

def create_empty_block_result(block_size):
    """
    Create an empty block_result dictionary.

    Parameters:
        block_size: int - Number of stations in the block

    Returns:
        dict: Empty result container with all variables initialized
    """
```

Listing 4.3: Zarr Schema Functions

4.3 Calendar Tables Module (`calendar_tables.py`)

4.3.1 Overview

Provides calendar conversion functions for the Persian calendar system.

4.3.2 Key Functions

```
def is_persian_leap_year(year):
    """Check if a Persian year is a leap year."""

def get_persian_month_days(month, year):
    """Get the number of days in a Persian month."""
```

```

def get_shamsi_doy(year, month, day):
    """Calculate the day-of-year for a Persian date."""

def build_doy_table():
    """Build the day-of-year lookup table."""

def build_calendar_tables():
    """Build all calendar tables, using calendar.txt if available."""
```

Listing 4.4: Calendar Functions

4.4 Runtime Tables Module (`runtime_tables.py`)

4.4.1 Overview

Builds tables needed at runtime, independent of the calendar system.

4.4.2 Key Functions

```
def build_window_table():
    """Build the window table for all days."""
    # For each day i, returns [i-2, i-1, i, i+1, i+2] with wrapping at
    boundaries

def build_file_map(zarr_base):
    """Build a map from (year, month) to Zarr file path."""

def build_runtime_tables(zarr_base):
    """Build all runtime tables."""
```

Listing 4.5: Runtime Tables Functions

4.5 I/O Pipeline Module (`io_pipeline/*.py`)

4.5.1 `read_month_files.py`

```
def read_month_files(block_start, block_size, file_map, year_list):
    """
    Read data from Zarr files for a block of stations.

    Parameters:
        block_start: int - Starting station index
        block_size: int - Number of stations to read
        file_map: dict - Mapping from (year, month) to file path
        year_list: list - All years in the analysis period

    Returns:
        dict: {(year_idx, month, var_idx): ndarray}
    """
    # For each year and month, opens the corresponding Zarr file
    # Slices the required station range
```

```
    # Extracts temperature variables and applies scaling for tmean
    # Returns a dictionary of raw arrays
```

Listing 4.6: read_month_files.py

4.5.2 assemble_block.py

```
def assemble_block(data_dict, doy_table, block_size, year_list):
    """
    Assemble raw data into a 4D array.

    Parameters:
        data_dict: dict - Raw data from read_month_files
        doy_table: ndarray - Day-of-year lookup table
        block_size: int - Number of stations
        year_list: list - All years

    Returns:
        ndarray: Shape (block_size, N_YEARS, N_DAYS, N_VARS)
    """
    # Initializes array with NaN values
    # For each year, month, and variable:
    #   Maps days to correct day-of-year positions
    #   Copies data to the assembled array
    # Missing days remain as NaN
```

Listing 4.7: assemble_block.py

4.5.3 validate_block.py

```
def validate_block(block_data, block_start, block_size, strict=True):
    """
    Validate assembled block data against the data contract.

    Parameters:
        block_data: ndarray - The assembled block
        block_start: int - Starting station index
        block_size: int - Number of stations
        strict: bool - Whether to raise exceptions on validation failure

    Returns:
        dict: Validation report with status, warnings, and errors
    """
    # Checks:
    #   - Shape correctness
    #   - Data type
    #   - Contiguous memory layout
    #   - NaN ratio
    #   - Infinite values
    #   - Per-station data completeness
```

Listing 4.8: validate_block.py

4.6 Numerical Engine Module (`numerical_engine/*.py`)

4.6.1 `window_engine.py`

```
def extract_window_values_fast(station_data, window_table, var_idx):
    """
    Extract window values for all days of a station.

    Parameters:
        station_data: ndarray - Shape (N_YEARS, N_DAYS, N_VARS)
        window_table: list - Window indices for each day
        var_idx: int - Variable index to extract

    Returns:
        list: 366 entries, each containing clean values array or None
    """
    # For each day:
    #   Extracts values from window days
    #   Removes NaN values
    #   Returns array if length >= MIN_VALID_VALUES, else None
```

Listing 4.9: window_engine.py

4.6.2 `distributions.py`

This module implements the core distribution fitting functions using Numba JIT compilation.

Normal Distribution

```
@njit
def fit_normal_full_numba(data):
    """
    Fit a Normal distribution to data.

    Returns:
        tuple: (mean, std, loglikelihood, AICc, BIC, n_parameters)
    """
    # Calculates mean and standard deviation
    # Computes log-likelihood
    # Calculates AICc and BIC
```

Listing 4.10: Normal Distribution Fitting

Skew-Normal Distribution

```
@njit
def skewness_to_alpha(skew):
    """Convert sample skewness to skew-normal shape parameter alpha"""

@njit
def fit_skewnorm_full_numba(data):
    """
    Fit a Skew-Normal distribution using method of moments.
```

```
    Returns:
        tuple: (alpha, loc, scale, loglikelihood, AICc, BIC, n_parameters)
    """
    # Estimates initial parameters from moments
    # Computes log-likelihood
    # Calculates AICc and BIC
```

Listing 4.11: Skew-Normal Fitting

Bimodal Normal Distribution

```
@njit
def fit_bimodal_full_numba(data):
    """
    Fit a Bimodal Normal distribution using moment-based estimation.

    Returns:
        tuple: (w1, mu1, sigma1, mu2, sigma2, loglikelihood, AICc, BIC,
    n_parameters)
    """
    # Estimates 5 parameters from moments:
    #   w1: mixing weight of component 1
    #   mu1, sigma1: mean and std of component 1
    #   mu2, sigma2: mean and std of component 2
    # Uses kurtosis to estimate separation between modes
```

Listing 4.12: Bimodal Fitting

Pearson Type III Distribution

```
@njit
def fit_pearson3_full_numba(data):
    """
    Fit a Pearson Type III (Gamma) distribution.

    Returns:
        tuple: (shape, scale, location, loglikelihood, AICc, BIC,
    n_parameters)
    """
    # Uses method of moments with skewness adjustment
    # Applies Wilson-Hilferty transformation for initial estimates
    # Computes log-likelihood via Gamma PDF
```

Listing 4.13: Pearson Type III Fitting

Master Fitting Function

```
@njit
def fit_all_distributions_numba(data):
    """
    Fit all four distributions and select the best.

    Returns:
        ndarray: 30-element array containing:
```

```
            [0]  best_dist (0=Normal, 1=Skew, 2=Bimodal, 3=Pearson, -1=
    failed)
            [1]  mean
            [2]  std
            [3]  skewness
            [4]  median
            [5]  count
            [6]  normal_p1 (mean)
            [7]  normal_p2 (std)
            [8]  normal_loglik
            [9]  normal_aicc
            [10] normal_bic
            [11] skew_p1 (alpha)
            [12] skew_p2 (loc)
            [13] skew_p3 (scale)
            [14] skew_loglik
            [15] skew_aicc
            [16] skew_bic
            [17] bimodal_p1 (w1)
            [18] bimodal_p2 (mu1)
            [19] bimodal_p3 (sigma1)
            [20] bimodal_p4 (mu2)
            [21] bimodal_p5 (sigma2)
            [22] bimodal_loglik
            [23] bimodal_aicc
            [24] bimodal_bic
            [25] pearson_p1 (shape)
            [26] pearson_p2 (scale)
            [27] pearson_p3 (location)
            [28] pearson_loglik
            [29] pearson_aicc
    """
    # Fits all 4 distributions
    # Selects best using AICc
    # Returns 30-element array with all parameters
```

Listing 4.14: fit_all_distributions_numba

4.6.3 analyze_station.py

```
def analyze_station(station_data, window_table, var_indices):
    """
    Analyze a single station for multiple variables.

    Parameters:
        station_data: ndarray - Shape (N_YEARS, N_DAYS, N_VARS)
        window_table: list - Window indices
        var_indices: list - Variable indices to analyze

    Returns:
        dict: Result dictionary with 87 keys (29 per variable)
    """
    # For each variable index:
    #   Extracts window values using extract_window_values_fast
    #   For each day:
    #     Fits distributions using fit_distribution
```

```
    #     Populates result dictionary with parameters
```

Listing 4.15: analyze_station.py

4.6.4 merge_results.py

```
def merge_station_result(block_result, station_result, local_idx):
    """Merge a station's results into the block result."""

def create_and_merge_results(block_data, window_table, var_indices):
    """
    Process all stations in a block and merge results.

    Returns:
        dict: Complete block result with 87 variables
    """
    # Iterates through stations
    # Calls analyze_station for each
    # Merges results into block_result
```

Listing 4.16: merge_results.py

4.7 Result Pipeline Module (result_pipeline/*.py)

4.7.1 validate_result.py

```
def validate_result(block_result, block_start, block_size, strict=True):
    """
    Validate analysis results before writing.

    Returns:
        dict: Validation report
    """
    # Checks:
    #   - Shape of each variable
    #   - Data type matches schema
    #   - best_dist contains only valid codes
    #   - count values in valid range
    #   - No negative standard deviations
    #   - No infinite values in loglik/AICc/BIC
```

Listing 4.17: validate_result.py

4.7.2 write_block.py

```
def write_block(root, block_result, block_start, block_end):
    """Write block results to Zarr store."""

def write_block_safe(root, block_result, block_start, block_end, validate=
    True):
    """Write block results with optional validation."""
```

Listing 4.18: write_block.py

4.8 Orchestrator Module (`orchestrator/*.py`)

4.8.1 `process_block.py`

```
class FitError(Exception): pass
class DataError(Exception): pass
class IOError(Exception): pass

def process_block(block_start, block_end, block_idx, file_map, doy_table,
                  window_table, year_list, root, var_indices,
   last_checkpoint_station=None):
    """
    Process a single block of stations.

    Returns:
        dict: Processing statistics for the block
    """
    # 1. Load data using read_month_files and assemble_block
    # 2. Validate loaded data
    # 3. Analyze using create_and_merge_results
    # 4. Validate results
    # 5. Write results
    # 6. Save checkpoint
    # 7. Return statistics
```

Listing 4.19: process_block.py

4.9 Monitoring Module (`monitoring/*.py`)

4.9.1 `benchmark.py`

```
def run_benchmark(load_func, analyze_func, write_func, test_sizes=None):
    """
    Run benchmark tests to determine optimal block size.

    Returns:
        dict: Benchmark results with recommended block size
    """
    # For each test size:
    #   Measures load, analyze, write times
    #   Monitors RAM usage
    #   Counts failed fits
    #   Calculates stations per second
    # Returns best performing size

def print_benchmark_report(report):
    """Pretty-print benchmark results."""
```

Listing 4.20: benchmark.py

4.9.2 `checkpoint.py`

```
def save_checkpoint(block_idx, station_idx, elapsed=None):
    """Save checkpoint information to file."""

def load_checkpoint():
    """Load checkpoint information from file."""

def delete_checkpoint():
    """Delete the checkpoint file after successful completion."""
```

Listing 4.21: checkpoint.py

4.9.3 logger.py

```
def setup_logger():
    """Set up logging configuration."""

logger = setup_logger()
```

Listing 4.22: logger.py

Chapter 5

Data Contract

5.1 Dynamic Distribution Architecture

Starting from version 2.1, the engine supports a **dynamic distribution architecture** that allows adding new distributions without reprocessing the raw data.

5.1.1 Key Concepts

- **Window Cache**: During the initial processing, the engine stores window values (5-day windows for each station and day) in a separate Zarr store (`window_cache.zarr`).
- **Modular Distributions**: Each distribution is defined as a dictionary in `zarr_schema.py` with its parameters, code, and fitting function.
- **Add Distribution Script**: The `add_distribution.py` script reads the window cache, fits a new distribution, and updates the output Zarr with new variables and `best_dist`.

5.1.2 Adding a New Distribution

To add a new distribution (e.g., Logistic):

1. Add the distribution to `DISTRIBUTIONS` in `zarr_schema.py`
2. Implement the fitting function in `distributions.py` and add it to `FIT_FUNCS`
3. Run `python add_distribution.py --dist logistic`

This process does **not require reading the raw input files again**.

5.1.3 Benefits

- No need to reprocess all data for each new distribution
- Saves significant time and computational resources
- Enables experimentation with different distribution families
- Maintains backward compatibility with existing outputs

5.2 Load Phase Contract

5.2.1 Input

- `block_start`: integer – Starting station index
- `block_size`: integer – Number of stations
- `file_map`: dict – `(year, month)` → `file_path`
- `year_list`: list – All years in analysis period
- `doy_table`: ndarray – Shape `(N_YEARS, 13, 32)`

5.2.2 Output

- `block_data`: ndarray – Shape `(block_size, N_YEARS, N_DAYS, N_VARS)`
- Data type: `float32`
- Memory layout: C-contiguous
- Missing values: `NaN`

5.3 Window Phase Contract

5.3.1 Input

- `station_data`: ndarray – Shape `(N_YEARS, N_DAYS, N_VARS)`
- `window_table`: list – 366 entries

5.3.2 Output

- List of 366 entries
- Each entry: `ndarray` or `None`
- Non-None arrays contain valid values (NaNs removed)
- Length $\geq$ `MIN_VALID_VALUES` $= 5$

5.4 Analyze Phase Contract

5.4.1 Input

- `station_data`: ndarray – Shape `(N_YEARS, N_DAYS, N_VARS)`

5.4.2 Output

- Dict with 87 keys
- Each key: ndarray – Shape `(N_DAYS,)`
- Types: `int32` for `best_dist` and `count`
- Types: `float32` for all others

5.5 Write Phase Contract

5.5.1 Input

- `block_result`: dict – 87 variables
- Each variable: Shape `(N_DAYS, block_size)`

5.5.2 Output

- Zarr store with 87 variables
- Shape: `(N_DAYS, n_stations)`
- Coordinated with `day_of_year` and `point` dimensions

5.6 Constants

Table 5.1: Key Constants

Constant	Value	Description
`WINDOW_SIZE`	5	Days in window (2 before + target + 2 after)
`MAX_VALUES_PER_FIT`	155	`N_YEARS * WINDOW_SIZE`
`MIN_VALID_VALUES`	5	Minimum valid values for fitting
`N_OUTPUTS`	87	Number of output variables
`VALID_BEST_DIST`	{-1,0,1,2,3}	Valid distribution codes

5.7 Error Types

Table 5.2: Error Types

Error Type	Description
`IOError`	Read/write errors (file not found, permission denied)
`DataError`	Invalid data (shape mismatch, missing values, infinite values)
`FitError`	Distribution fitting failed (insufficient data, numerical issues)

Chapter 6

Mathematical Background

6.1 Distributions

6.1.1 Normal Distribution

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Parameters: μ (location), $\sigma > 0$ (scale)

6.1.2 Skew-Normal Distribution

$$f(x) = \frac{2}{\omega}\phi\left(\frac{x-\xi}{\omega}\right)\Phi\left(\alpha\frac{x-\xi}{\omega}\right)$$

where ϕ is the standard Normal PDF, Φ the standard Normal CDF, ξ location, $\omega > 0$ scale, α shape.

6.1.3 Bimodal Normal Distribution

$$f(x) = w_1\mathcal{N}(x;\mu_1,\sigma_1^2) + w_2\mathcal{N}(x;\mu_2,\sigma_2^2)$$

with $w_1 + w_2 = 1$, $0 \leq w_i \leq 1$, $\sigma_i > 0$.

6.1.4 Pearson Type III Distribution

$$f(x) = \frac{1}{\Gamma(\alpha)\beta^\alpha}(x-\gamma)^{\alpha-1}\exp\left(-\frac{x-\gamma}{\beta}\right)$$

with shape $\alpha > 0$, scale $\beta > 0$, location γ.

6.2 Model Selection Criteria

6.2.1 AIC (Akaike Information Criterion)

$$\mathrm{AIC} = 2k - 2\ln(\hat{L})$$

6.2.2 AICc (Corrected AIC)

$$\text{AICc} = \text{AIC} + \frac{2k(k+1)}{n-k-1}$$

6.2.3 BIC (Bayesian Information Criterion)

$$\text{BIC} = k\ln(n) - 2\ln(\hat{L})$$

6.3 Bimodality Metrics

6.3.1 Ashman's D

$$D = \frac{|\mu_2 - \mu_1|}{\sqrt{\frac{\sigma_1^2+\sigma_2^2}{2}}}$$

6.3.2 Bimodality Coefficient

$$\text{BC} = \frac{\text{skewness}^2 + 1}{\text{kurtosis} + 1}$$

6.3.3 Overlap Coefficient

$$\text{OVL} = \int_{-\infty}^{\infty} \min\{f_1(x), f_2(x)\}\, dx$$

6.4 Window-Based Estimation

For each day d, the engine extracts values from days $d-2$ to $d+2$ (total 5 days) across all years:

$$\text{Window}_d = \{x_{y,t} : y \in \text{years}, t \in [d-2, d+2]\}$$

with wrapping at boundaries.

Chapter 7

Execution Flow

7.1 Main Function

```
def main():
    # 1. Load configuration
    # 2. Get station information from input data
    # 3. Build calendar and runtime tables
    # 4. Run benchmark (if enabled)
    # 5. Create Zarr output store
    # 6. Load checkpoint (if exists)
    # 7. Process blocks in a loop
    #    - For each block: load, validate, analyze, write, checkpoint
    # 8. Finalize Zarr store
    #    - Add coordinates and metadata
    #    - Consolidate metadata
    # 9. Delete checkpoint
    # 10. Log completion
```

Listing 7.1: Main Execution Flow

7.2 Block Processing Flow

1. Load data for block
2. Validate loaded data
3. Analyze each station
4. Validate results
5. Write results to Zarr
6. Save checkpoint

Figure 7.1: Block Processing Steps

7.3 Recovery Flow

When a failure occurs:

1. On startup, check for checkpoint file
2. If checkpoint exists, load the last successful block
3. Resume processing from the next station in that block
4. Continue with the main processing loop

Chapter 8

Input Data Format

8.1 Input Zarr Structure

Each monthly Zarr file must contain:

Table 8.1: Input Zarr Variables

Variable	Shape	Description
`tmin`	`(days, points)`	Minimum temperature (raw)
`tmax`	`(days, points)`	Maximum temperature (raw)
`tmean`	`(days, points)`	Mean temperature (raw, scaled by 10)
`stationid`	`(points,)`	Station identifiers
`lat`	`(points,)`	Latitude
`lon`	`(points,)`	Longitude
`elev`	`(points,)`	Elevation

8.2 Data Scaling

The `tmean` data is stored with a scaling factor of 10 (actual = stored / 10). The engine automatically handles this scaling during loading.

8.3 Missing Data

Missing values are stored as `NaN` and are handled appropriately.

Chapter 9

Output Format

9.1 Zarr Structure

The output Zarr store contains:

```
/
--- tmin_best_dist        (day_of_year, point)
--- tmin_mean             (day_of_year, point)
--- tmin_std              (day_of_year, point)
--- tmin_skewness         (day_of_year, point)
--- tmin_median           (day_of_year, point)
--- tmin_count            (day_of_year, point)
--- tmin_normal_p1        (day_of_year, point)
--- tmin_normal_p2        (day_of_year, point)
--- tmin_normal_loglik    (day_of_year, point)
--- tmin_normal_aicc      (day_of_year, point)
--- tmin_normal_bic       (day_of_year, point)
--- tmin_skew_p1          (day_of_year, point)
--- tmin_skew_p2          (day_of_year, point)
--- tmin_skew_p3          (day_of_year, point)
--- tmin_skew_loglik      (day_of_year, point)
--- tmin_skew_aicc        (day_of_year, point)
--- tmin_skew_bic         (day_of_year, point)
--- tmin_bimodal_p1       (day_of_year, point)
--- tmin_bimodal_p2       (day_of_year, point)
--- tmin_bimodal_p3       (day_of_year, point)
--- tmin_bimodal_p4       (day_of_year, point)
--- tmin_bimodal_p5       (day_of_year, point)
--- tmin_bimodal_loglik   (day_of_year, point)
--- tmin_bimodal_aicc     (day_of_year, point)
--- tmin_bimodal_bic      (day_of_year, point)
--- tmin_pearson_p1       (day_of_year, point)
--- tmin_pearson_p2       (day_of_year, point)
--- tmin_pearson_p3       (day_of_year, point)
--- tmin_pearson_loglik   (day_of_year, point)
--- tmin_pearson_aicc     (day_of_year, point)
--- ... (same for tmean and tmax)
--- day_of_year           (day_of_year,)
--- point                 (point,)
--- stationid             (point,)
--- lat                   (point,)
--- lon                   (point,)
--- elev                  (point,)
```

```
--- ...
```

Listing 9.1: Output Zarr Structure

9.2 Metadata

Global attributes:

Table 9.1: Global Attributes

Attribute	Description
`description`	”Climatology 366 days - Station-wise architecture”
`source`	Years range, e.g., ”Years 1369-1399, Window ±2 days”
`architecture`	”Station-wise, Block-oriented”
`version`	Engine version
`n_outputs`	Number of output variables
`min_valid_values`	Minimum values required for fitting
`var_defs`	List of all variable definitions

Chapter 10

Performance Optimization

10.1 Block Size Selection

The optimal block size depends on available RAM, CPU cache, disk I/O speed, and number of cores. The benchmarking system automatically evaluates block sizes from 500 to 3000 and selects the one with the highest stations-per-second rate.

10.2 Parallel Processing

When `use_parallel` is enabled, the engine uses `multiprocessing` or `concurrent.futures`; each worker processes a separate block; the `cores` parameter controls the number of workers.

10.3 Memory Management

1. **Block-based processing**: Data is processed in blocks to limit memory usage
2. **Garbage collection**: Active memory is reclaimed after each block
3. **NaN handling**: Missing values use `float32` to reduce memory footprint
4. **Chunked Zarr**: Output is stored in chunks for efficient access

10.4 Performance Tips

1. Use SSD storage for both input and output data
2. Increase the number of cores if available
3. Use smaller block sizes if RAM is limited
4. Disable validation for production runs to reduce overhead
5. Run on a dedicated machine without memory-intensive applications

Chapter 11

Troubleshooting

11.1 Common Errors and Solutions

11.1.1 FileNotFoundError

```
FileNotFoundError: [Errno 2] No such file or directory: 'path/to/file'
```

Listing 11.1: FileNotFoundError

Solution:

1. Verify the file path in `config.yaml`
2. Check that the file exists and is readable
3. Use absolute paths to avoid ambiguity

11.1.2 MemoryError

```
MemoryError: Unable to allocate array
```

Listing 11.2: MemoryError

Solution:

1. Reduce the `block_size` in `config.yaml`
2. Disable validation for lower memory usage
3. Use `use_parallel: false`
4. Increase system RAM or use a machine with more memory

11.1.3 KeyError

```
KeyError: 'variable_name'
```

Listing 11.3: KeyError

Solution:

1. Check that the variable exists in the input Zarr files

2. Verify variable naming matches the schema
3. Ensure the `VARS` list in `config.yaml` is correct

11.1.4 ImportError

```
ImportError: No module named 'module_name'
```

Listing 11.4: ImportError

Solution:

1. Install missing dependencies
2. Check Python environment (use `which python`)
3. Use `pip install -r requirements.txt`

11.1.5 IndentationError

```
IndentationError: unexpected indent
```

Listing 11.5: IndentationError

Solution:

1. Check that Python files use consistent indentation (4 spaces)
2. Avoid mixing tabs and spaces
3. Use an editor with Python indentation support

11.2 Logging

The engine logs all major events to:

- **Console**: Real-time progress information
- **Log file**: `climatology.log` in the output directory

Log levels: `DEBUG`, `INFO` (default), `WARNING`, `ERROR`.

11.3 Debugging Tips

1. Set `logging.level: DEBUG` for more verbose output
2. Check the log file for detailed error traces
3. Run a small test with `block_size: 100` to verify setup
4. Use Python's interactive debugger (`pdb`) for deep troubleshooting
5. Verify input data quality before large-scale processing

Chapter 12

Support and Community

12.1 Citation

If you use ClimateProcessingEngine in your research, please cite:

```
cff-version: 1.2.0
title: ClimateProcessingEngine
authors:
  - family-names: Kazemi
    given-names: Amin Fazl
version: v1.0.0
date-released: 2026-07-24
repository-code: https://github.com/AminFazlKazemi/ClimateProcessingEngine
license: MIT
```

Listing 12.1: CITATION.cff

12.2 Repository

https://github.com/AminFazlKazemi/ClimateProcessingEngine

12.3 License

MIT License.

12.4 Contact

For questions, issues, or contributions:

- Email: aminfazlkazemi@gmail.com
- GitHub Issues: https://github.com/AminFazlKazemi/ClimateProcessingEngine/issues

Appendix A

Full Variable Definitions

Table A.1: Complete Variable Definitions

Variable Name	Dtype	Description
tmin Variables		
tmin_best_dist	int32	Best distribution code (-1=failed, 0=Normal, 1=Skew, 2=Bimodal, 3=Pearson)
tmin_mean	float32	Mean temperature
tmin_std	float32	Standard deviation
tmin_skewness	float32	Skewness
tmin_median	float32	Median
tmin_count	int32	Number of valid values
tmin_normal_p1	float32	Normal: mean
tmin_normal_p2	float32	Normal: standard deviation
tmin_normal_loglik	float32	Normal: log-likelihood
tmin_normal_aicc	float32	Normal: AICc
tmin_normal_bic	float32	Normal: BIC
tmin_skew_p1	float32	Skew-Normal: alpha (shape)
tmin_skew_p2	float32	Skew-Normal: location
tmin_skew_p3	float32	Skew-Normal: scale
tmin_skew_loglik	float32	Skew-Normal: log-likelihood
tmin_skew_aicc	float32	Skew-Normal: AICc
tmin_skew_bic	float32	Skew-Normal: BIC
tmin_bimodal_p1	float32	Bimodal: w1 (mixing weight)
tmin_bimodal_p2	float32	Bimodal: mu1 (mean of component 1)
tmin_bimodal_p3	float32	Bimodal: sigma1 (std of component 1)
tmin_bimodal_p4	float32	Bimodal: mu2 (mean of component 2)
tmin_bimodal_p5	float32	Bimodal: sigma2 (std of component 2)
tmin_bimodal_loglik	float32	Bimodal: log-likelihood
tmin_bimodal_aicc	float32	Bimodal: AICc
tmin_bimodal_bic	float32	Bimodal: BIC
tmin_pearson_p1	float32	Pearson Type III: shape
tmin_pearson_p2	float32	Pearson Type III: scale
tmin_pearson_p3	float32	Pearson Type III: location
tmin_pearson_loglik	float32	Pearson Type III: log-likelihood

Variable Name	Dtype	Description
tmin_pearson_aicc	float32	Pearson Type III: AICc
tmean Variables (same structure)		
... (30 variables with tmean_ prefix)		
tmax Variables (same structure)		
... (30 variables with tmax_ prefix)		

Appendix B

Configuration File Reference

```yaml
# Paths
paths:
  calendar_file: K:/Temp/needed/calendar.txt      # Path to Persian calendar file
  checkpoint_file: checkpoint.csv                 # Checkpoint file name
  output_dir: I:/climatology_366_rolling          # Output directory
  output_zarr_name: climatology_stationwise_final.zarr  # Output file name
  zarr_base: K:/.../zarr_yearly_monthly           # Input Zarr base directory

# Time Range
years:
  start: 1369                                     # First year (Persian calendar)
  end: 1399                                       # Last year (inclusive)
days: 366                                         # Days per year (365 or 366)
window_days: 2                                    # Window size +/- days

# Processing
block_size: 1000                                  # Stations per block
cores: 6                                          # CPU cores for parallel
use_parallel: false                               # Enable parallel processing

# Variables
variables:
- tmin                                           # Variables to process
- tmean
- tmax
fit_variable_index: 1                             # Index for fitting (0=tmin, 1=tmean, 2=tmax)

# Validation
validation:
  validate_after_load: true                       # Validate after loading
  validate_before_write: true                     # Validate before writing
  validate_every_n_blocks: 5                      # Validate every N blocks

# Logging
logging:
  level: INFO                                     # Log level (DEBUG, INFO,
```

```
    WARNING, ERROR)
  log_file: climatology.log                          # Log file name
  log_timestamps: true                               # Include timestamps in
    logs

# Benchmark
benchmark:
  enabled: true                                      # Enable benchmarking
  run_on_first_block: true                           # Run benchmark on first
    run
  test_block_sizes:                                  # Block sizes to test
  - 500
  - 1000
  - 1500
  - 2000
  - 3000
```

Listing B.1: Full config.yaml with Descriptions

Appendix C

Example Usage

C.1 Running the Engine

```
cd K:/gozareshha/dr vazife/140504 - qc temp/climatology/climatology_engine
python main.py
```

Listing C.1: Basic Execution

C.2 Resuming After Interruption

Simply run `python main.py` again. The engine will automatically detect the checkpoint and resume.

C.3 Starting Fresh

To start from scratch:

```
rm checkpoint.csv
rm -rf I:/climatology_366_rolling/climatology_stationwise_final.zarr
python main.py
```

Listing C.2: Starting Fresh

C.4 Analyzing Results

After the engine completes:

```
cd ../..
python analyze_distributions1.py
```

Listing C.3: Analyzing Results

C.5 Example: Custom Configuration

```yaml
# config.yaml
paths:
  output_dir: /data/climate/output
  output_zarr_name: my_climate_results.zarr
  zarr_base: /data/climate/input
years:
  start: 1370
  end: 1400
block_size: 2000
cores: 8
use_parallel: true
variables:
- tmin
- tmax
```

Listing C.4: Custom Configuration Example

Appendix D

Glossary

Table D.1: Glossary of Terms

Term	Definition
AIC	Akaike Information Criterion – measures model quality balancing fit and complexity.
AICc	Corrected AIC – with small-sample correction.
BIC	Bayesian Information Criterion – alternative to AIC with stronger penalty for complexity.
Block	A contiguous group of stations processed together for efficiency.
Checkpoint	A saved state that allows resuming after interruption.
Doy	Day of Year – a number from 1 to 365 (or 366) representing the day.
Fit	The process of estimating distribution parameters from data.
GEV	Generalized Extreme Value – a family of distributions for extreme events.
Log-likelihood	The logarithm of the likelihood function, used in model selection.
NaN	Not a Number – used to represent missing data.
Numba	A Python library for just-in-time compilation that accelerates numerical computations.
Pearson III	A three-parameter Gamma distribution, also known as a shifted Gamma.
Skew-Normal	A distribution that extends the Normal with a shape parameter for asymmetry.
Window	A set of days around a target day used for estimation (e.g., ± 2 days).
Zarr	A Python library for chunked, compressed, N-dimensional arrays.
Zarr Store	A directory containing Zarr data and metadata.